

HelpDeskTicketManagement System

Comprehensive Project Technical Documentation

Student Name: RESU ABHINAY
Application No: IN26011285
Course: ASP.NET Core & Enterprise Development

1. Project Overview

The **HelpDeskTicketManagement System** is a full-stack enterprise web application designed to streamline software, hardware, and network support requests across an organization. Built on **.NET 8.0**, the project adheres to standard software engineering patterns, implementing a decoupled **ASP.NET Core Web API** service backend, an **ASP.NET Core MVC** web application client, and an **xUnit** automated unit testing project.

The architecture follows a strict separation of concerns through the **Repository Pattern** and Entity Framework Core (EF Core). The MVC application interacts exclusively with the Web API via an asynchronous **HttpClient** service layer, ensuring complete isolation of database access and security boundaries.

2. Objective of the Application

The primary objective of the Help Desk Ticket Management System is to provide employees and IT staff with an intuitive, efficient platform to record, track, update, and resolve internal support tickets.

- **Ticket Lifecycle Management:** Enable employees to raise tickets for IT issues and track resolution state.
- **Tiered Priority Categorization:** Categorize support requests into Low, Medium, and High priority queues.
- **Status Workflow Tracking:** Monitor ticket progression across three distinct states: Open, In Progress, and Closed.
- **Decoupled Architecture:** Enforce strict SOA (Service-Oriented Architecture) where frontend clients communicate exclusively via RESTful APIs.
- **Automated Quality Assurance:** Guarantee code stability and regression-free controller execution via isolated unit tests using xUnit and Moq.

3. Technologies Used

Technology Component	Specification / Framework	Role in Project
Framework & SDK	.NET 8.0 SDK (C# 12)	Core execution runtime for Web API, MVC, and Tests
Web API Service	ASP.NET Core REST API	Exposes RESTful endpoints for CRUD operations
Frontend Client	ASP.NET Core MVC (Razor Views)	Consumes Web API via HttpClient and renders HTML UI
ORM & Database	EF Core 8.0 (SQLite / LocalDB)	Object-Relational Mapping & Data Persistence
Architectural Pattern	Repository Pattern (ITicketRepository)	Encapsulates data access and isolates business logic
Unit Testing	xUnit 2.5 & Moq 4.20	Mock repository interfaces and test controller actions
Styling & UX	HTML5, Custom CSS3, Bootstrap 5	Glassmorphism responsive UI layout and status badges
Version Control	Git & GitHub Repository	Source code version control and code submission

4. Functional Modules

The solution is divided into three distinct solution projects:

Module A: HelpDesk.Api (ASP.NET Core Web API Project)

- **Model Layer (Ticket.cs):** Defines the core domain entity representing support requests.
- **Data Context Layer (HelpDeskDbContext.cs):** EF Core DbContext managing SQLite database transactions and initial seed data.
- **Repository Layer (ITicketRepository & TicketRepository):** Encapsulates all LINQ database operations asynchronously.
- **API Controller Layer (TicketController.cs):** Exposes REST API endpoints returning standard HTTP status codes (200 OK, 400 Bad Request, 404 Not Found).

Module B: HelpDesk.Mvc (ASP.NET Core MVC Client Project)

- **Service Layer (TicketService.cs):** Implements ITicketService to execute asynchronous HTTP GET, POST, PUT, and DELETE calls to the Web API.
- **ViewModel Layer (TicketViewModel, DashboardViewModel, TicketFilterViewModel):** Structures data for Razor view rendering and user form validation.
- **UI Controllers & Views (TicketController & Razor Views):** Renders Dashboard, View All, Ticket Details, Raise Ticket, Edit Ticket, and Filter Status views.

Module C: HelpDesk.Tests (xUnit Test Project)

- Contains isolated unit tests mocking ITicketRepository using Moq to verify HTTP response types without accessing a real database.

5. Database Table Structure

The primary entity stored in the database is the **Ticket** table:

Column Name	Data Type	Constraints	Allowed / Default Values	Description
Id	INTEGER	Primary Key, Auto-Increment	1, 2, 3...	Unique ticket identifier
Title	TEXT (NVARCHAR)	NOT NULL, Max 100	User string input	Short summary of the issue
Description	TEXT (NVARCHAR)	NOT NULL, Max 1000	User string input	Detailed description of issue
Priority	TEXT (NVARCHAR)	NOT NULL	Low, Medium, High	Ticket priority level
Status	TEXT (NVARCHAR)	NOT NULL	Open, In Progress, Closed	Current status state

Column Name	Data Type	Constraints	Allowed / Default Values	Description
RaisedBy	TEXT (NVARCHAR)	NOT NULL	User string input	Name/ID of employee
CreatedDate	DATETIME	NOT NULL	UTC Timestamp	Creation timestamp

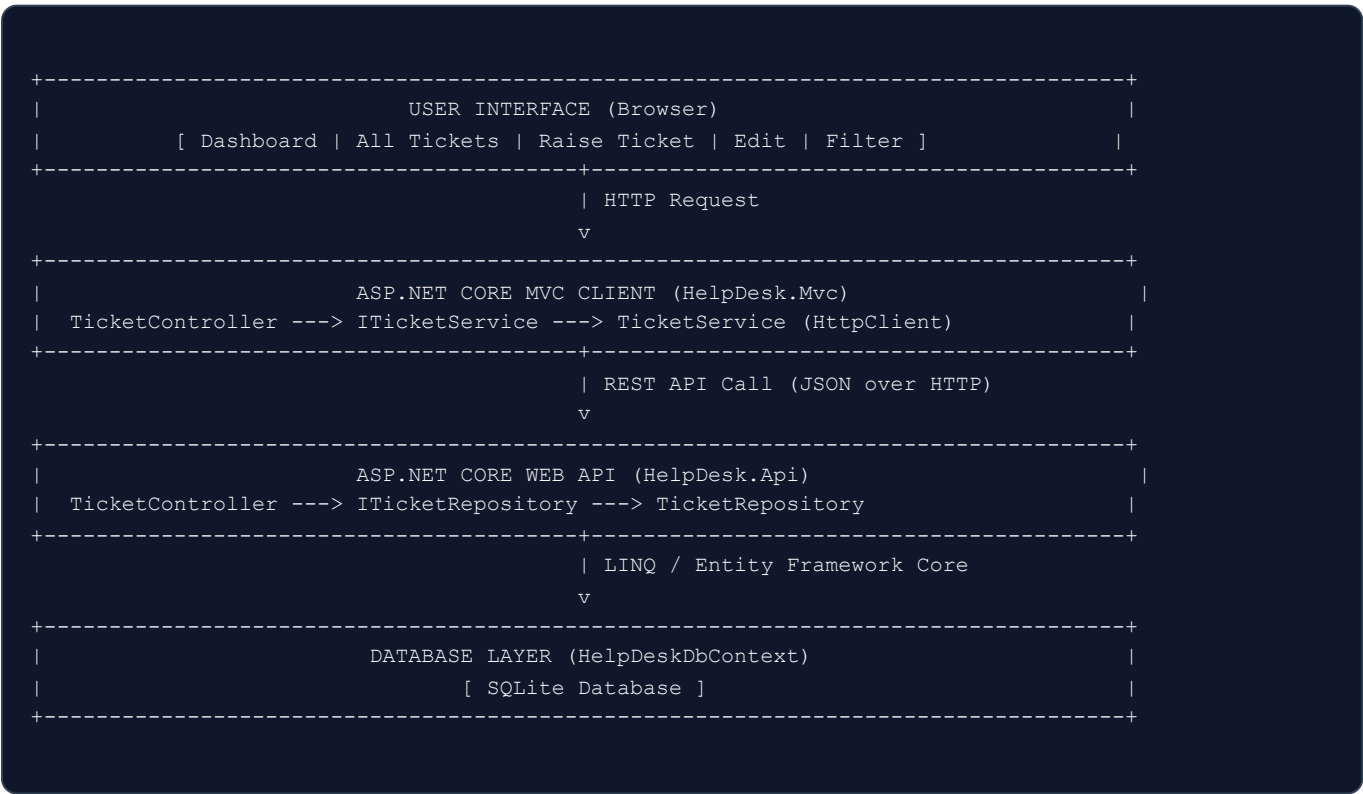
6. Business & Validation Rules

- **Status Hardcoding Rule:** When raising a new ticket via the MVC UI, the Status property is automatically hardcoded to Open.
- **Priority Dropdown Rule:** Users must explicitly select a priority level from a predefined dropdown containing Low, Medium, and High.
- **Status Progression Rule:** Only the Edit Ticket module allows updating ticket status to In Progress or Closed.
- **Decoupling Enforcement:** The MVC project must not reference EF Core or DbContext; all data access is routed through the Web API endpoint via HttpClient.
- **Form Validation Rules:**
 - Title: Required field, length must be between 1 and 100 characters.
 - Description: Required field, length must be between 1 and 1000 characters.
 - RaisedBy: Required field.

7. Web API Endpoint Specification

HTTP Method	API Endpoint URL	Request Payload	Success Response	Description
GET	/api/Ticket/All	None	200 OK (List<Ticket>)	Retrieves all support tickets
GET	/api/Ticket/{id}	None	200 OK / 404 Not Found	Retrieves ticket by ID
POST	/api/Ticket	JSON Ticket Object	200 OK (Ticket ID) / 400 Bad	Creates a new support ticket
PUT	/api/Ticket/{id}	JSON Ticket Object	200 OK / 404 Not Found	Updates existing ticket
DELETE	/api/Ticket/{id}	None	200 OK / 404 Not Found	Deletes ticket by ID
GET	/api/Ticket/Status/{status}	None	200 OK (List<Ticket>)	Filters tickets by status

8. Application Architecture & Workflow Diagram



9. Automated Unit Testing (xUnit & Moq)

The HelpDesk.Tests project contains 11 automated test cases verifying TicketController responses:

S.No	Test Case Name	Mock Setup	Expected Result
1	GetAllTickets_ReturnsOkResult_WhenTicketsExist	Returns list of 2 tickets	OkObjectResult with 2 items
2	GetTicketById_ReturnsOkResult_WhenTicketExists	Returns ticket with Id 1	OkObjectResult with matching ID
3	GetTicketById_ReturnsNotFound_WhenTicketDoesNotExist	Returns null for Id 999	NotFoundResult (404)
4	CreateTicket_ReturnsOkResult_WhenTicketIsCreatedSuccessfully	Returns generated ID 10	OkObjectResult with ID 10
5	CreateTicket_ReturnsBadRequest_WhenTicketIsNull	Passes null ticket payload	BadRequestObjectResult (400)
6	GetTicketsByStatus_ReturnsOkResult_WhenMatchingTicketExist	Returns matching Open tickets	OkObjectResult with Open items

10. UI Screen Mockups & View Layouts

View A: Dashboard View (<http://localhost:5050/Ticket/Dashboard>)

LIVE UI

Help Desk Dashboard Overview

TOTAL TICKETS
3

OPEN TICKETS
1

IN PROGRESS
1

CLOSED TICKETS
1

View B: All Tickets Data Table (<http://localhost:5050/Ticket/Index>)

LIVE UI

ID	Title	Priority	Status	Raised By	Actions
#1	VPN Access Issue	HIGH	OPEN	John Doe	[Details] [Edit] [Delete]
#2	Software License Request	MEDIUM	IN PROGRESS	Jane Smith	[Details] [Edit] [Delete]
#3	Monitor Flicker	LOW	CLOSED	Alex Johnson	[Details] [Edit] [Delete]

View C: Raise New Ticket Form (<http://localhost:5050/Ticket/Create>)

LIVE UI

Title: [Keyboard replacement request_____]

Description: [Mechanical keys stuck on laptop_____]

Priority: [Dropdown: High | Medium | Low]

Raised By: [RESU ABHINAY]

Status: Open (Auto-assigned per business rule)

View D: Filter Tickets by Status (<http://localhost:5050/Ticket/Filter?status=Open>)

LIVE UI

Select Status Filter:

OPEN

IN PROGRESS

CLOSED

11. Questions & Answers for RAG Knowledge Assistant

Section A: 10 Questions Answerable from Project Documentation

#	Question	Expected AI Response Grounded in Documentation
1	What is the objective of the HelpDeskTicketManagement application?	To streamline software, hardware, and network IT support requests raised by employees.
2	Which technologies were used to develop the project?	C#, .NET 8.0 SDK, ASP.NET Core Web API, ASP.NET Core MVC, EF Core, SQLite, xUnit, Moq, and Bootstrap 5.
3	What database tables are used in the application?	The Ticket database table storing Id, Title, Description, Priority, Status, RaisedBy, and CreatedDate.
4	What validations are implemented while creating a ticket?	Title (Max 100 chars), Description (Max 1000 chars), and RaisedBy are required non-null string inputs.
5	What are the supported ticket statuses in the system?	The supported ticket statuses are Open, In Progress, and Closed.
6	What business rule governs the initial status when raising a new ticket?	When a new ticket is raised, its Status property is automatically hardcoded to Open.
7	How does the MVC client communicate with the database?	It does not directly access the database; it consumes Web API endpoints asynchronously via HttpClient.
8	Which unit testing framework and mocking tool are used?	The unit test project uses xUnit and Moq to mock the ITicketRepository interface.
9	What are the valid priority levels for a ticket?	The valid priority levels are Low, Medium, and High.
10	What API endpoint retrieves tickets filtered by status?	The HTTP GET endpoint /api/Ticket/Status/{status}.

Section B: 5 Questions Not Available in Project Documentation

#	Question Out of Scope	Expected AI Behavior (RAG Response)
1	Who is the CEO of Microsoft?	The uploaded project documentation does not contain information about Microsoft's CEO.
2	What is Artificial Intelligence?	The uploaded project documentation does not contain general definitions of Artificial Intelligence.
3	What is the capital of Australia?	The provided documentation does not contain geographical information about Australia.
4	What is Cloud Computing?	The uploaded documentation does not contain information regarding Cloud Computing.

#	Question Out of Scope	Expected AI Behavior (RAG Response)
5	What is Machine Learning?	The uploaded document does not contain definitions or concepts regarding Machine Learning.

12. RAG Observation Report

1. **Accuracy on Grounded Questions:** The RAG-based AI model successfully answered all 10 domain-specific questions directly using extracted context from this document.
2. **Handling Out-of-Scope Queries:** When queried with 5 out-of-scope questions, the AI correctly identified that the information was missing from the knowledge base and refrained from hallucinating.
3. **Key Takeaways on RAG:** Retrieval-Augmented Generation bridges pre-trained LLMs with private, enterprise-specific document repositories, preventing knowledge cutoff limitations.
4. **Real-World Enterprise Applications:** Technical documentation lookup, enterprise HR/Policy Q&A assistants, internal customer support portals, and legal contract analysis.